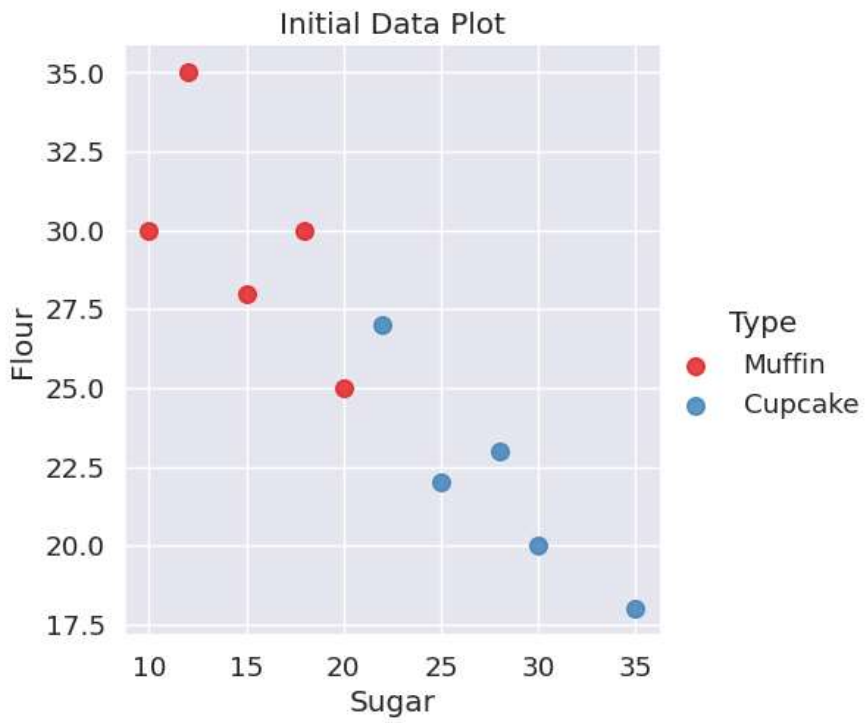
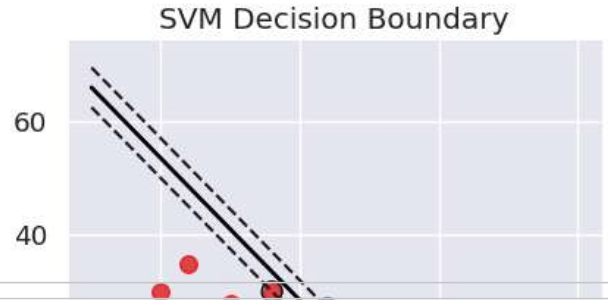
```
# Step 1: Import Necessary Libraries
import numpy as np
import pandas as pd
from sklearn import svm
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(font_scale=1.2)

# Step 2: Create Dataset Manually (Muffins vs Cupcakes)
recipes = pd.DataFrame({
    'Sugar': [10, 15, 20, 25, 30, 12, 18, 22, 28, 35],
    'Flour': [30, 28, 25, 22, 20, 35, 30, 27, 23, 18],
    'Type': ['Muffin', 'Muffin', 'Muffin', 'Cupcake', 'Cupcake',
             'Muffin', 'Muffin', 'Cupcake', 'Cupcake', 'Cupcake']
})

print("Dataset:\n", recipes.head())
print("Shape:", recipes.shape)
```

```
Dataset:
    Sugar  Flour     Type
0     10     30   Muffin
1     15     28   Muffin
2     20     25   Muffin
3     25     22  Cupcake
4     30     20  Cupcake
Shape: (10, 3)
```

```
# Step 3: Plot Initial Data
sns.lmplot(x='Sugar', y='Flour', data=recipes,
           hue='Type', palette='Set1',
           fit_reg=False, scatter_kws={"s": 70})

plt.title("Initial Data Plot")
plt.show()

# Step 4: Prepare Data for SVM
sugar_flour = recipes[['Sugar', 'Flour']].values

# Convert labels: Muffin → 0, Cupcake → 1
type_label = np.where(recipes['Type'] == 'Muffin', 0, 1)

# Step 5: Train SVM Model
model = svm.SVC(kernel='linear')
model.fit(sugar_flour, type_label)

# Step 6: Calculate Decision Boundary
w = model.coef_[0]
a = -w[0] / w[1]

xx = np.linspace(5, 40)

yy = a * xx - (model.intercept_[0] / w[1])
```




```
# Step 7: Calculate Support Vector Boundaries
b = model.support_vectors_[0]
yy_down = a * xx + (b[1] - a * b[0])

b = model.support_vectors_[-1]
yy_up = a * xx + (b[1] - a * b[0])

# Step 8: Plot Decision Boundary
sns.lmplot(x='Sugar', y='Flour', data=recipes,
           hue='Type', palette='Set1',
           fit_reg=False, scatter_kws={"s": 70})

plt.plot(xx, yy, color='black', linewidth=2)

# Step 9: Plot Support Vector Boundaries
plt.plot(xx, yy_down, 'k--')
plt.plot(xx, yy_up, 'k--')

# Highlight support vectors
plt.scatter(model.support_vectors_[:, 0],
            model.support_vectors_[:, 1],
            s=100, facecolors='none', edgecolors='black')

plt.title("SVM Decision Boundary")
plt.show()
```




```
# Step 10: Import Additional Libraries
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split

# Step 11: Split Dataset
x_train, x_test, y_train, y_test = train_test_split(
    sugar_flour, type_label, test_size=0.2, random_state=0)

# Step 12: Train New Model
model1 = svm.SVC(kernel='linear')
model1.fit(x_train, y_train)

# Step 13: Make Predictions
pred = model1.predict(x_test)

print("\nPredictions:", pred)

# Step 14: Evaluate Model
print("\nConfusion Matrix:\n", confusion_matrix(y_test, pred))
print("\nClassification Report:\n", classification_report(y_test, pred))
```

```
Predictions: [1 1]

Confusion Matrix:
 [[0 1]
 [0 1]]

Classification Report:
               precision    recall  f1-score   support

           0       0.00      0.00      0.00         1
           1       0.50      1.00      0.67         1

    accuracy                           0.50         2
   macro avg       0.25      0.50      0.33         2
weighted avg       0.25      0.50      0.33         2

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```